
Hourly Energy Forecasting

RNN vs LSTM Performance Analysis

TL;DR: Motivation, Main Idea & Results

MOTIVATION

Demand forecasting is critical for many use cases. Classical methods (ARIMA or Linear regressing) are interpretable but struggle with **complex temporal patterns** and rely on manual features and has lots of assumptions.

This project explores if **Deep Learning** can automatically capture these patterns and generate better forecast.

MAIN IDEA

We compare 4 configurations of **Simple RNN** and **LSTM** architectures built in PyTorch using the DOM energy dataset from Kaggle.

Model are RNN and LSTM, baseline and enhanced version with cyclical time embeddings.

The task is one-step-ahead hourly forecasting: given the last 24 hours, predict the next hour's MW demand.

RESULTS

Model	RMSE	R ²	MAPE
RNN Base	215.46	0.992	1.44%
LSTM Base	219.30	0.991	1.44%
RNN + Feat	204.17	0.993	1.37%
LSTM + Feat	160.86	0.995	1.07%

Conclusion: **LSTM with cyclical features** is the clear winner, delivering a **25% RMSE reduction** over the baseline. The combination of architecture and feature engineering provides the largest gain.

Literature Review on Related Methods

SIMPLE RNN

Designed for sequential data, it updates a **hidden state** at each step to carry information forward. Reads 24-hour history for next-hour forecasts.

Core weakness: **Vanishing gradients** make it hard to learn dependencies beyond 10–20 steps.

LSTM ARCHITECTURE

Fixes the RNN's weakness using a **cell state** and three learned **gates** (forget, input, output) to control information flow.

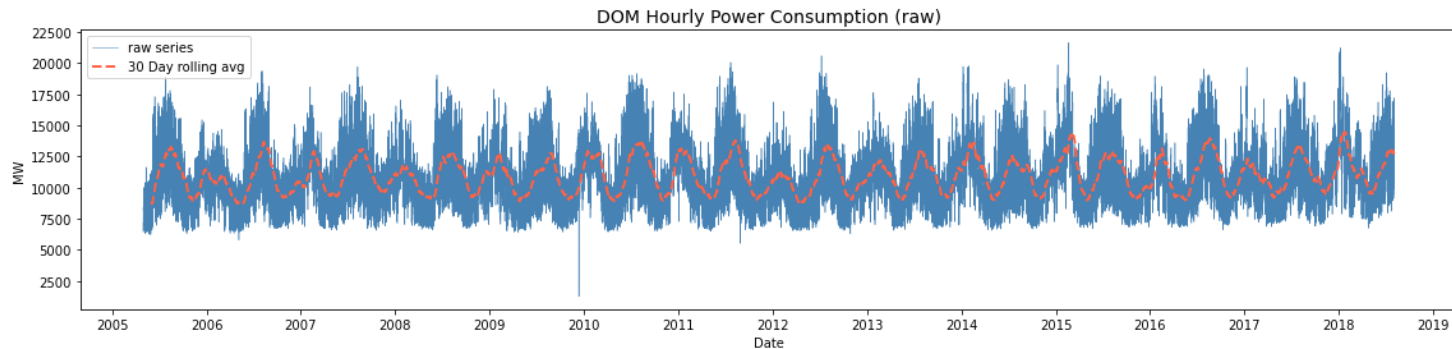
Effectively retains relevant context over hundreds of steps, capturing **daily and weekly periodicities**.

SECTION 1

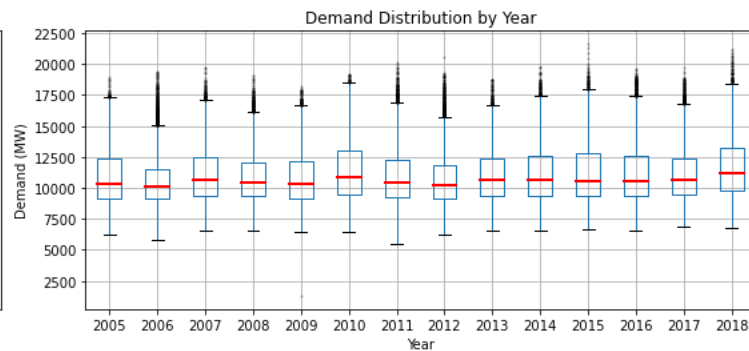
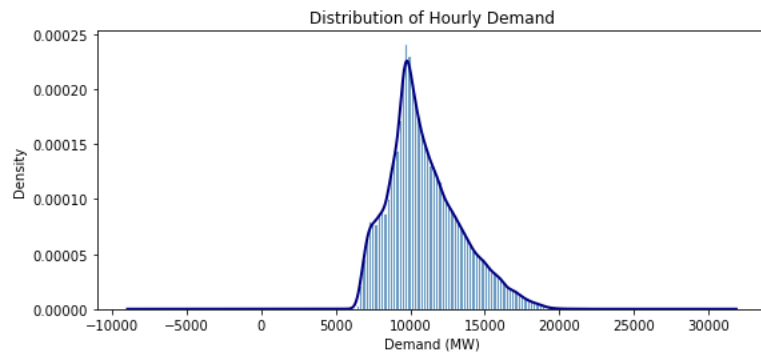
Exploratory Data Analysis (EDA)

Time Series Trends and Distribution

RAW DEMAND TREND



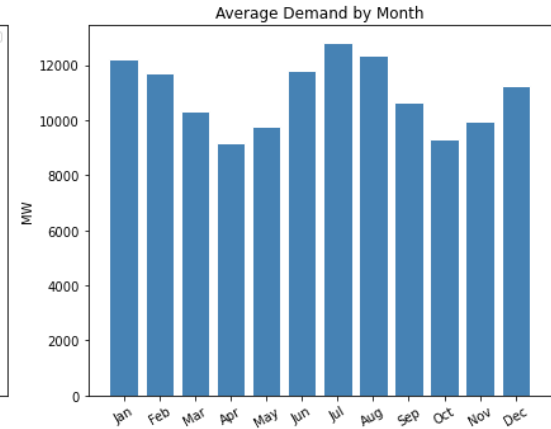
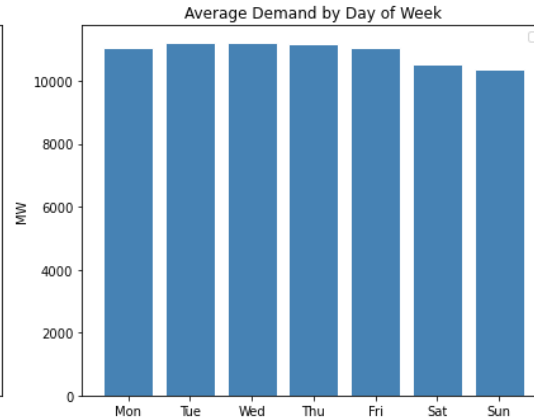
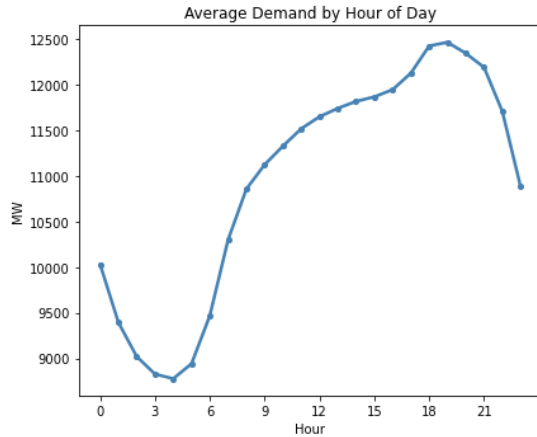
CONSUMPTION DISTRIBUTION



Seasonal Patterns

Analysis reveals **strong seasonality in daily, weekly, and monthly.**

Morning and evening hourly peak; weekends lowers than weekdays; winter and summer peak;



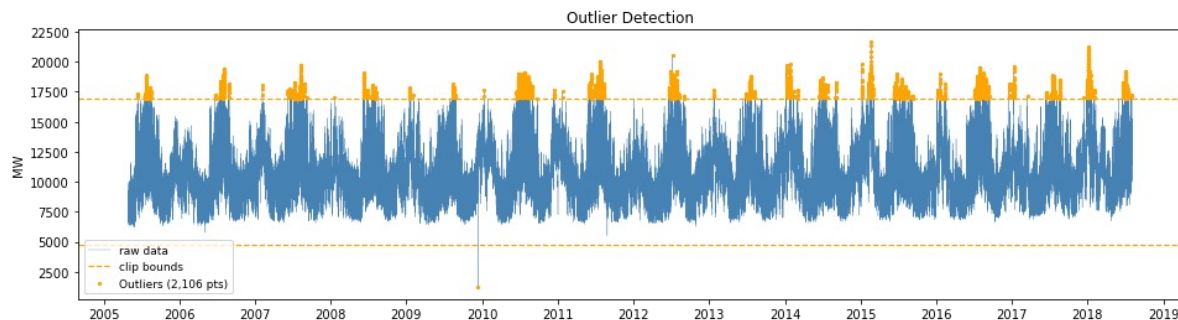
SECTION 2

Data Preprocessing

Cleaning and Time-Ordered Splitting

OUTLIER TREATMENT

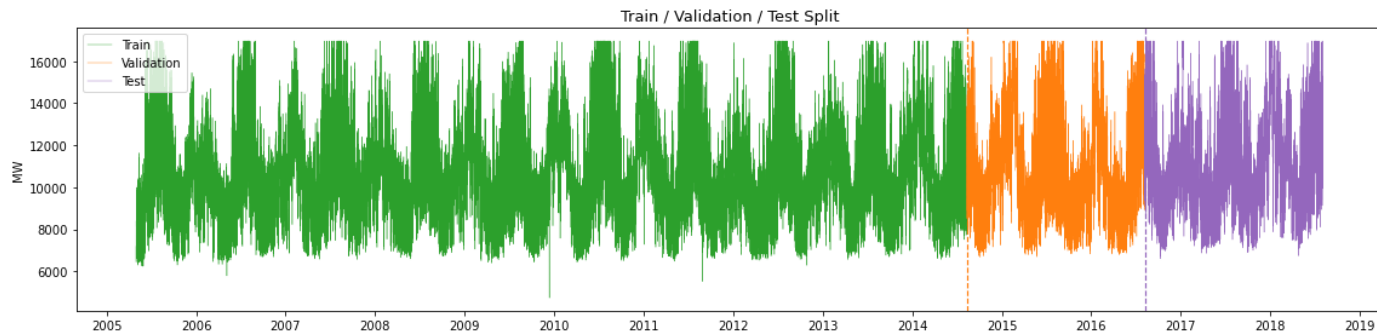
2,106 extreme points (1.81%) were detected using IQR and clipped to fence boundary



TIME SPLIT

Data is split by **time order** to prevent data leakage (70% training, 15% validation, 15% test).

Normalized by Min-Max to map value in $[0,1]$, and scaling parameters are fitted **only on training data**.



Engineering Cyclical Time Features

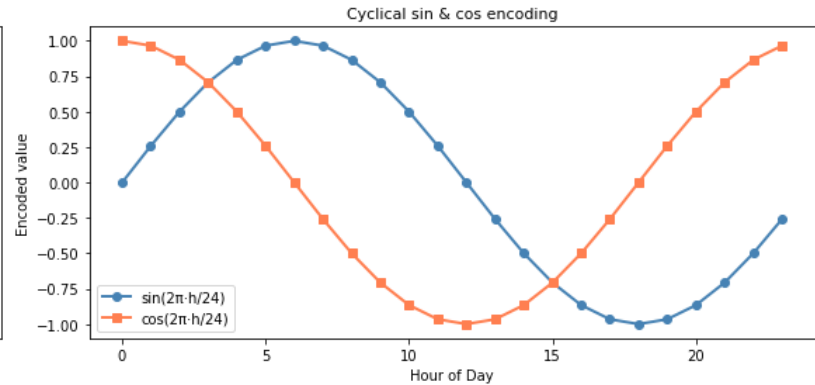
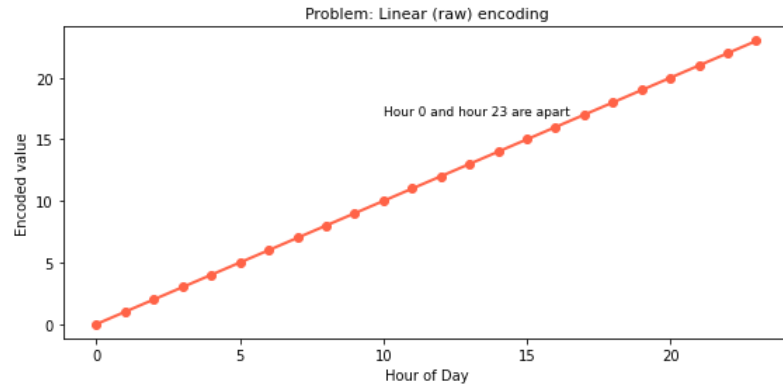
IDEA

Enrich feature set for training. Linear encoding (0-23) creates a **false distance** between hour 23 and hour 0; researched another way called cyclical embedding.

Project time onto a **2D circle** using sine and cosine transformations. This preserves the periodic nature of time, where period in 24, 7, 12

$$x_{\sin} = \sin(2 * \pi * t / \text{period})$$

$$x_{\cos} = \cos(2 * \pi * t / \text{period})$$



SECTION 3

Model Setup

Forecasting Architecture and Parameters

MODEL DESIGN

Sliding Window

Each sample uses a **24-hour input sequence** as a lag array to predict the single next-hour demand.

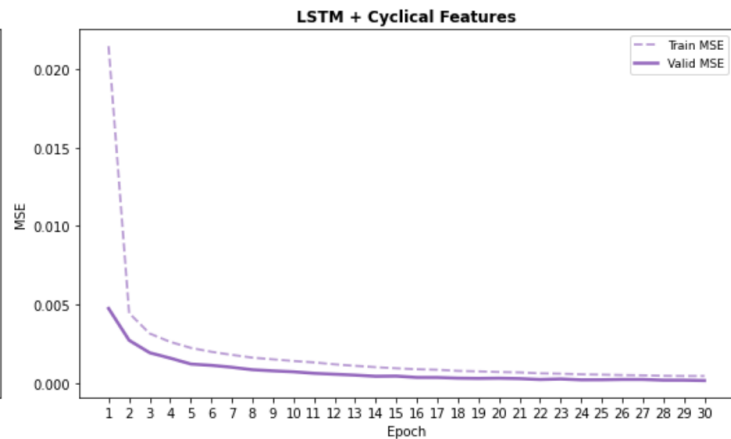
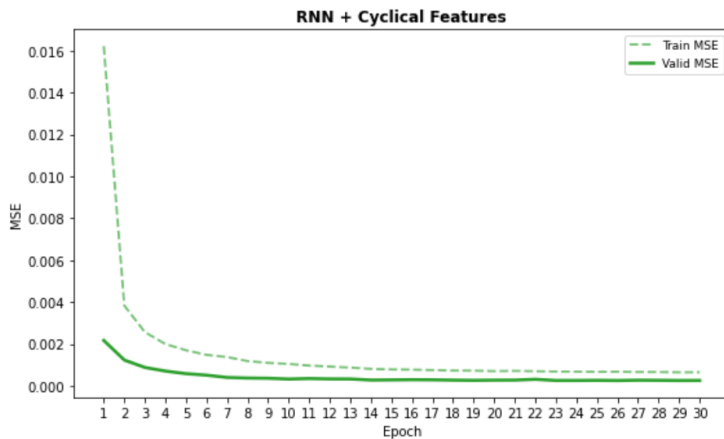
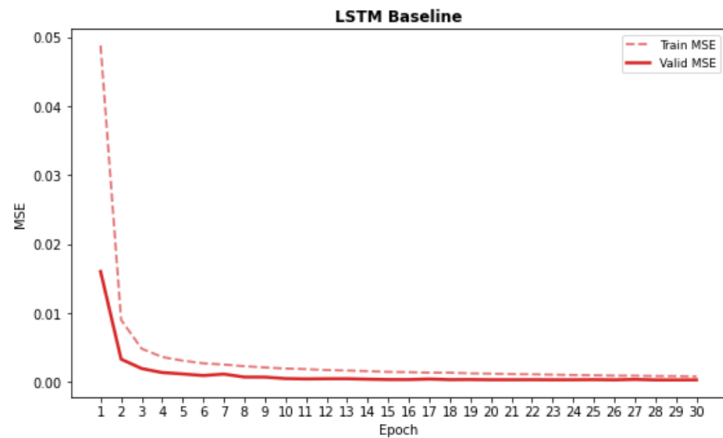
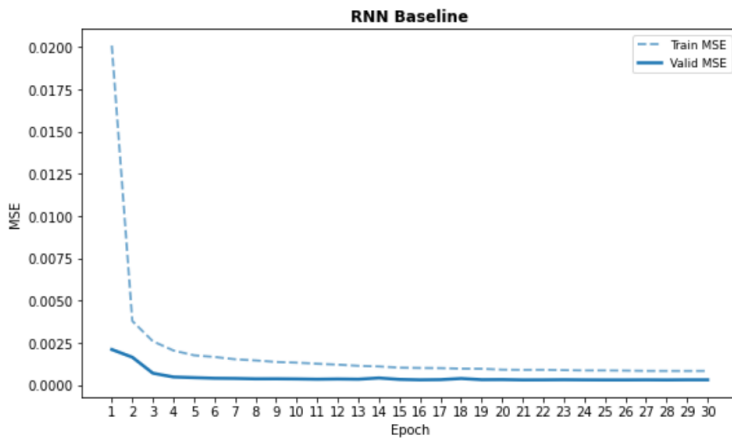
Architectures

Comparison between models **RNN** and **LSTM** (baseline vs enriched features) to evaluate the model capability.

HYPERPARAMETERS

Parameter	Value	Reason
hidden_size	64	Reasonable default for model to remember
num_layers	3	Depth allows model to learn temporal features
dropout	0.15	Light regularization for large data
optimizer	Adam	Adaptive learning rates, fast convergence
initial lr	1e-3	Standard default start point
epochs	30	Sufficient for convergence on this dataset
batch_size	512	Good for current data size

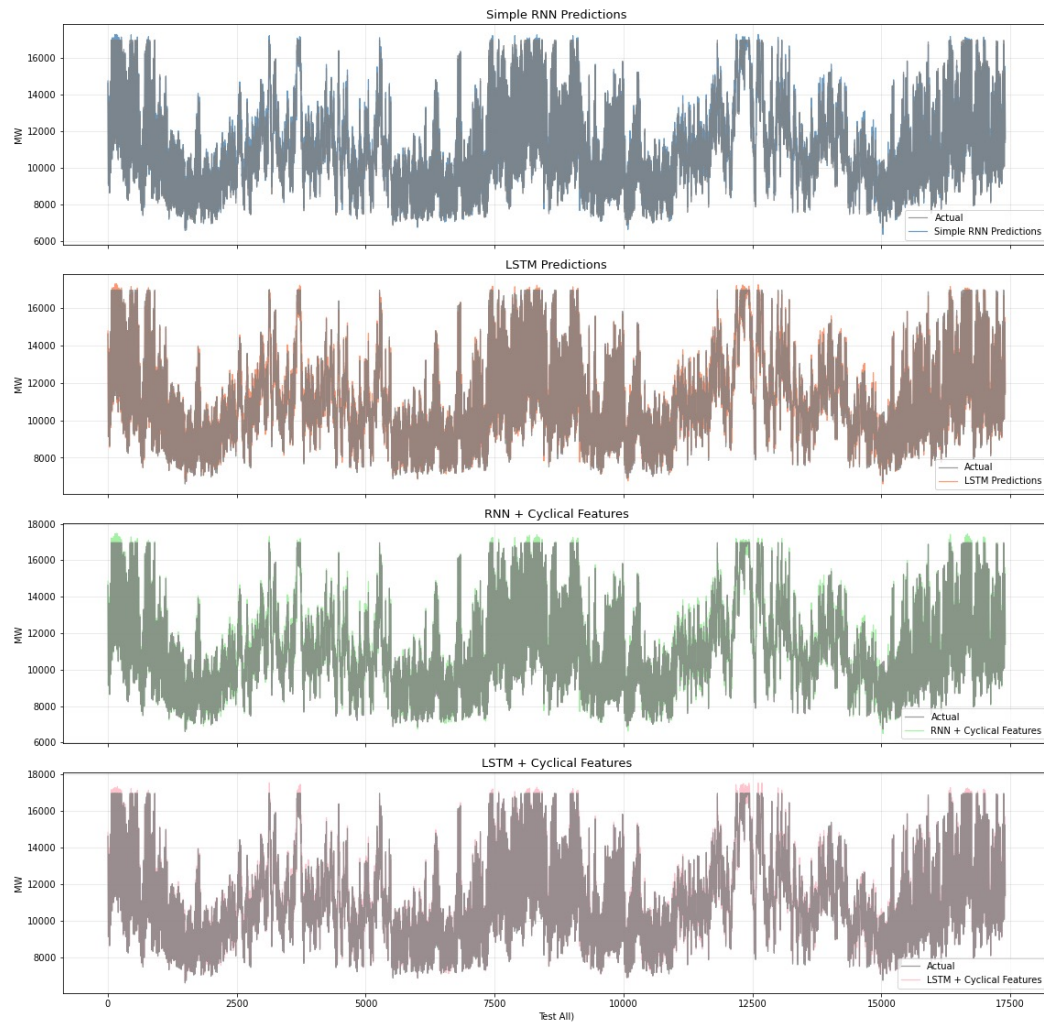
Training and Validation Loss Curves



SECTION 4

Results and Output Comparison

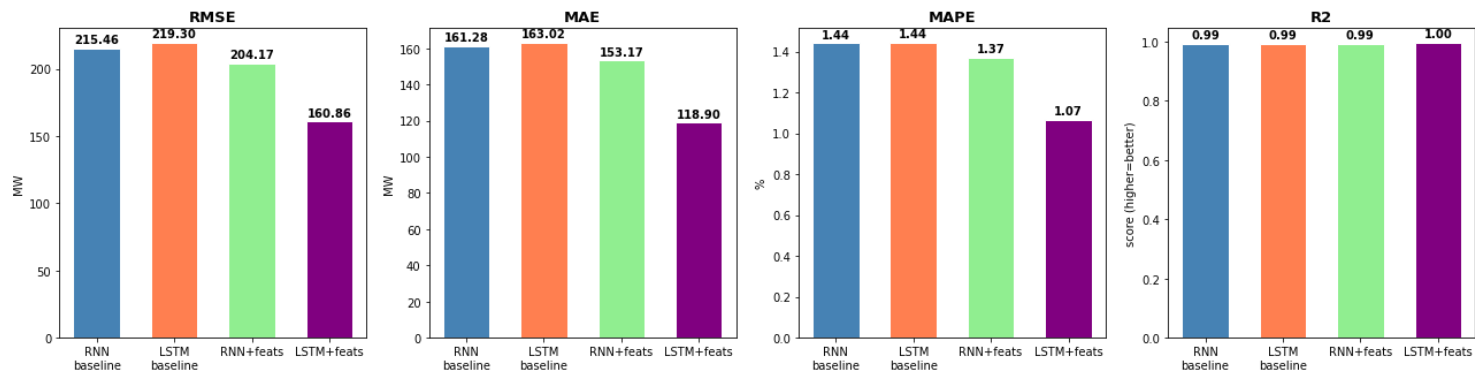
Forecast vs Actuals



Model Performance Metrics (Test Set)

MODEL CONFIGURATION	RMSE	MAE	R ²	MAPE
RNN Baseline	215.46	161.28	0.9917	1.44%
LSTM Baseline	219.30	163.02	0.9915	1.44%
RNN + Cyclical Features	204.17	153.17	0.9926	1.37%
LSTM + Cyclical Features	160.86	118.90	0.9954	1.07%

Test Set Metrics — All Model Configurations



Key Findings

LSTM wins

The **LSTM with cyclical time features** is the clear winner.

However, theoretically the LSTM base should outperform than RNN, but it turns out oppositely. The reason is LSTM needs more epochs to converge than RNN because it has 4x more parameters where 30 epochs might not enough.

FEATURE IMPACT

Temporal encoding (sin/cos) significantly reduces forecast error by providing the model with periodic context.

DATA QUALITY

Rigorous **outlier cleaning** and chronological splitting were essential for stable training and realistic evaluation.

Links and Reference

Result

- Github: https://github.com/ykklw/ai-hw-sp-2026-nn/tree/main/MRP_energy_fcst
- Youtube: <https://www.youtube.com/watch?v=ttesidRzbk0>

Reference

- Kaggle data: https://www.kaggle.com/datasets/robikscube/hourly-energy-consumption/data?select=DOM_hourly.csv
- Cyclical feature encoding: <https://www.kaggle.com/code/avanwyk/encoding-cyclical-features-for-deep-learning>
- PyTorch RNN docs: <https://pytorch.org/docs/stable/generated/torch.nn.RNN.html>
- PyTorch LSTM docs: <https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html>
- Sequence length discussion: <https://stackoverflow.com/questions/49573242/what-is-sequence-length-in-lstm>
- MinMaxScaler: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html>
- R^2 score: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.r2_score.html